How-To Library

Informatica

Relational Pushdown Optimization Best Practices for Developer Tool Mappings

© Copyright Informatica LLC 2016, 2021. Informatica LLC. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC. All other company and product names may be trade names or trademarks of their respective owners and/or copyrighted materials of such owners.

Abstract

To improve Developer tool mapping performance, use best practices when you configure mappings and apply relational pushdown optimization. Relational pushdown optimization causes the Data Integration Service to push transformation logic to a database. Pushdown optimization improves mapping performance as the source database can process transformation logic faster than the Data Integration Service. The Data Integration Service also reads less data from the source.

Supported Versions

- Informatica Data Services, Informatica Data Quality, Informatica Big Data Management 10.0

Table of Contents

Relational Pushdown Overview. 2
Pushdown Optimization Types. 2
Best Practices to Improve Mapping Performance. 3
Mapping Example. 4

Relational Pushdown Overview

Relational pushdown optimization causes the Data Integration Service to push transformation logic to a database. Pushdown optimization improves mapping performance as the source database can process transformation logic faster than the Data Integration Service. The Data Integration Service also reads less data from the source.

Relational pushdown optimization has the following advantages:

- It minimizes the data processed by the Data Integration Service.
- It utilizes the high performance of database resources that are available.
- It utilizes the hardware resources effectively and efficiently.

The performance of relational pushdown optimization depends on the performance of the database. If the database is not properly tuned, the relational pushdown may not be advantageous.

Pushdown Optimization Types

You can configure the following types of relational pushdown optimization:

Source-side pushdown optimization

The Data Integration Service pushes as much transformation logic as possible to the source database. The Data Integration Service attempts to push the entire mapping logic or until it reaches a transformation that it cannot push to the source database.

Full pushdown optimization

The Data Integration Service attempts to push all transformation logic to target database. The entire mapping logic is converted to an SQL query and is pushed to the database if the source and target schema is same.

Best Practices to Improve Mapping Performance

Consider the following best practices while configuring a mapping to improve performance:

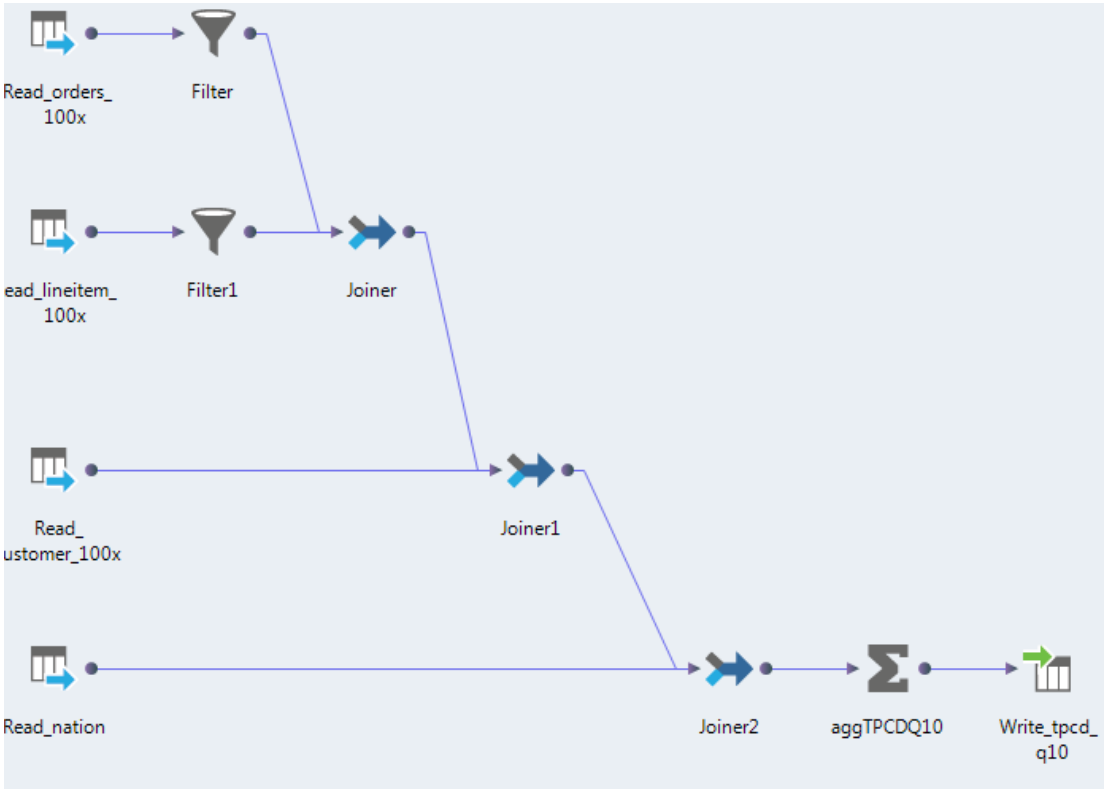
- Avoid using Decimal (38,0) ports in the mapping. Decimal (38,0) ports add an additional CAST function in the pushdown SQL.
- Maintain the same data type, precision and scale across the ports from source to transformations to target. Any change will result additional CAST statements in the pushdown SQL.
- Avoid using a very large precision. For example, avoid specifying a port with a large precision of 1000 bytes or higher precision.
- When you use an Aggregator, Expression, Filter, Joiner, Sorter, or Union transformation, pushing transformation logic to databases might improve performance by up to three times.
- When you push Lookup transformation logic to databases, disable Null Comparison. If you enable Null Comparison, mappings with multiple Lookup transformations might slow down performance. In the Data Integration Service properties, set the `Optimizer.PdoLookupIgnoreNullCompare` custom property to `true`.
- Push SQL transformation logic to databases. In the Data Integration Service properties, set the `Optimizer.PdoAllowSqlTxPushdown` custom property to `true`.
- When you use Update Strategy transformation in a mapping, push the logic for the Insert, Delete, Update or Merge operations to the database. Internal tests using TPCH schema-based mappings have yielded up to 22 times improvement in performance when compared the default configuration.

Note: You might need to apply custom properties in special cases. When you define a custom property, enter the property name and an initial value. Define custom properties only at the request of Informatica Global Customer Support.

Mapping Example

Consider a mapping with multiple relational sources writing to one relational target. The mapping contains Filter transformations, Joiner transformations, and an Aggregator transformation.

The following image shows the mapping:



When full pushdown optimization was configured for the mapping, the mapping took a few seconds to run. With default mapping configuration, the mapping took one hour to run.

Authors

Mohammed Morshed

Vidya Vasudevan